# MODUL AJAR — PERTEMUAN 12

**Jaringan Syaraf Tiruan & Deep Learning — Mengenali Angka Tulisan Tangan**

Sistem Cerdas (TI24317) · Prodi Teknologi Informasi · Politeknik Negeri Madiun

## I. IDENTITAS MODUL

| | |
|---|---|
| **Dosen Pengampu** | Muhammad Syaeful Fajar, S.Pd. Gr., M.Kom. |
| **Semester / SKS** | Gasal / 3 SKS |
| **Alokasi Waktu** | 8 JP × 50 menit = 400 menit (3 JP teori + 5 JP praktik) |
| **Sub-CPMK** | 0512 — Menerapkan teknologi cerdas untuk menyelesaikan masalah spesifik dalam industri sesuai standar industri. |
| **Prasyarat** | Modul 9–10 (pola fit/predict, train/test split). |
| **Indikator Capaian** | 1. Menjelaskan cara kerja neuron buatan.<br>2. Melatih model klasifikasi angka tulisan tangan.<br>3. Membaca akurasi dan menganalisis kesalahan prediksi. |
| **Bentuk Penilaian** | Notebook praktikum P12 (tugas, 2%) |

## II. PETUNJUK PENGGUNAAN MODUL

1. Hari ini kalian membangun teknologi di balik face unlock, filter wajah, dan ChatGPT — dalam versi paling kecilnya. Konsepnya sama, skalanya saja yang berbeda jutaan kali.
2. Praktikum inti memakai scikit-learn (sudah terpasang) dan dataset digits bawaan — tanpa unduhan, ringan untuk semua laptop. Keras/TensorFlow disediakan sebagai pengayaan opsional bagi yang laptopnya kuat.
3. Ikuti alur dramanya: neuron tunggal → perceptron belajar sendiri → GAGAL di XOR → hidden layer menyelamatkan → jaringan mengenali tulisan tangan. Kegagalan XOR itu sejarah nyata AI (1969) yang sempat membekukan riset JST puluhan tahun.
4. Tidak ada matematika turunan; cukup logika “geser bobot saat salah”.

## III. PETA KEGIATAN 8 JP



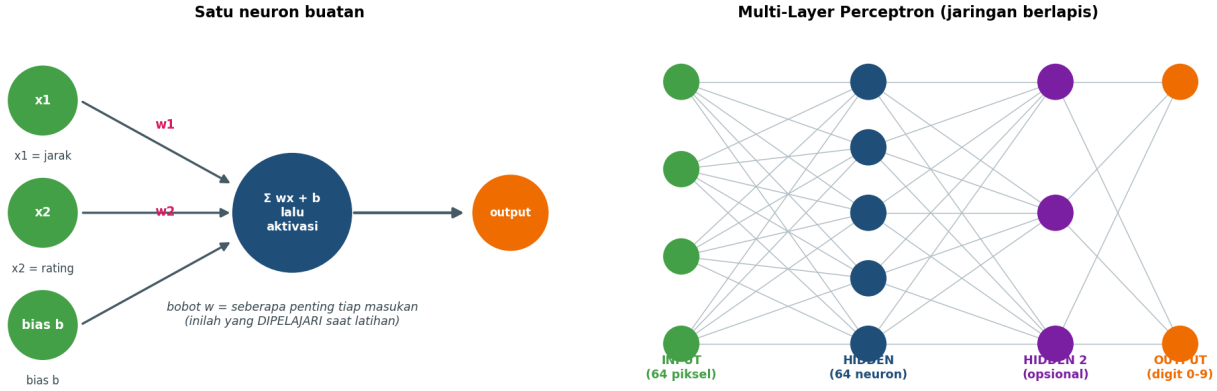

| Tahap | Waktu | Kegiatan |
|---|---|---|
| **Pendahuluan** | 30′ | Review P11; apersepsi: “bagaimana face unlock mengenali wajahmu dari jutaan piksel?” |
| **Teori 1** | 60′ | Dari neuron biologis ke neuron buatan; cara perceptron belajar. |

| Tahap | Waktu | Kegiatan |
|---|---|---|
| **Teori 2** | 60′ | Batas perceptron (kasus XOR); hidden layer; mengapa disebut deep learning. |
| **Praktik 1** | 70′ | Kegiatan 1–2: neuron manual + perceptron yang belajar sendiri (dan gagal di XOR). |
| **Praktik 2** | 90′ | Kegiatan 3: MLP mengenali angka tulisan tangan (digits 8×8). |
| **Praktik 3** | 60′ | Kegiatan 4: bedah salah prediksi + pengayaan Keras (opsional). |
| **Penutup** | 30′ | Kegiatan 5 dilanjutkan sebagai tugas; kesimpulan, logbook, info pertemuan 13. |

# BAGIAN A — TEORI (3 JP / 150 MENIT)

## A.1 🧠 Dari Neuron Biologis ke Neuron Buatan

Otak kalian berisi ±86 miliar neuron. Satu neuron menerima sinyal dari banyak neuron lain, menimbangnya (ada sinyal yang kuat pengaruhnya, ada yang lemah), lalu "menyala" bila total rangsangannya melewati ambang. Neuron buatan meniru persis mekanisme itu dengan aritmetika sederhana:




- Masukan (x) dikalikan bobot (w) masing-masing — bobot = seberapa penting masukan itu.
- Semua dijumlahkan bersama bias (b), lalu dilewatkan fungsi aktivasi yang memutuskan "menyala atau tidak".
- Belajar = mengatur bobot. Di P9 scikit-learn mencari koefisien regresi; di sini prinsipnya sama — hanya neuron-nya bisa disusun berlapis-lapis.

*Analogi: rapat pengambilan keputusan. Tiap anggota (masukan) punya suara, tetapi bobot suaranya berbeda; keputusan diambil bila total suara berbobot melewati ambang. Melatih jaringan = belajar dari pengalaman menyetel ulang bobot suara siapa yang layak didengar.*

## A.2 📚 Bagaimana Perceptron Belajar

Perceptron (Rosenblatt, 1958) adalah satu neuron dengan resep belajar yang sangat manusiawi: coba tebak → kalau salah, geser bobot sedikit ke arah yang benar → ulangi. Dalam pseudokode:

**Resep belajar perceptron (pseudokode)**

```
untuk setiap contoh (x, target):
    tebakan = 1 jika (w·x + b) > 0, selain itu 0
    error = target - tebakan
    w = w + laju * error * x      # salah? geser bobot
    b = b + laju * error
```

Sesederhana itu — dan untuk masalah tertentu, terbukti selalu menemukan jawaban. "Masalah tertentu" itulah kuncinya…

## A.3 💥 Tragedi XOR dan Kelahiran Deep Learning

Perceptron hanya mampu memisahkan dua kelas dengan satu garis lurus. Gerbang logika AND dan OR bisa; tetapi XOR (beda = 1, sama = 0) tidak mungkin dipisahkan satu garis — coba gambar sendiri titik (0,0),(1,1) vs (0,1),(1,0). Buku Minsky & Papert (1969) membuktikan batas ini dan riset JST membeku hampir 20 tahun — periode yang dijuluki AI Winter.

Jalan keluarnya: susun neuron berlapis. Lapisan tersembunyi (hidden layer) menggabungkan beberapa garis menjadi pemisah selentur apa pun. Makin banyak lapisan = makin “dalam” = deep learning. Lapisan awal mengenali pola sederhana (garis, lengkung), lapisan berikutnya merangkainya menjadi konsep (mata, hidung… wajah!). Itulah cara face unlock bekerja.

> 💡 **Konteks:** Sejarah singkat: perceptron (1958) → tragedi XOR (1969) → backpropagation menghidupkan MLP (1986) → deep learning meledak (2012, saat GPU cukup kuat). Yang kalian praktikkan hari ini merentang 60 tahun sejarah dalam satu notebook.

## A.4 🛠️ Ekosistem: dari sklearn sampai TensorFlow

- scikit-learn MLPClassifier — cukup untuk belajar konsep dan data kecil; dipakai di praktikum inti hari ini (ringan, tanpa instalasi).
- TensorFlow/Keras dan PyTorch — standar industri untuk jaringan besar (jutaan neuron, GPU); dicicipi di bagian pengayaan.
- Untuk gambar besar dipakai arsitektur khusus CNN; untuk teks dipakai transformer — fondasi LLM yang kalian temui minggu depan (P13).

> 🏭 **Penerapan industri:** JST versi besar (deep learning) kini menjadi mata industri: inspeksi visual otomatis cacat produk di lini manufaktur, pencocokan wajah untuk verifikasi identitas (e-KTP/perbankan digital), pembacaan citra medis — termasuk riset klasifikasi citra COVID-19 dengan CNN oleh peneliti Indonesia di jurnal TELKOMNIKA (terindeks Scopus, lihat Referensi) — hingga persepsi kendaraan otonom. Semuanya perbesar-besaran dari perceptron yang kalian bangun hari ini.

# BAGIAN B — PRAKTIKUM (5 JP / 250 MENIT)

## B.1 🎯 Tujuan Praktikum

1. Mahasiswa mampu mengimplementasikan neuron dan perceptron yang belajar dari nol (tanpa library).
2. Mahasiswa mampu menunjukkan kegagalan perceptron pada XOR dan keberhasilan MLP mengatasinya.
3. Mahasiswa mampu melatih MLP mengenali angka tulisan tangan dan membedah kesalahannya.

## B.2 🧰 Alat dan Bahan

| No | Alat / Bahan | Spesifikasi Minimal | Jumlah |
|---|---|---|---|
| 1 | Laptop / PC | Anaconda + Jupyter — scikit-learn sudah termasuk; dataset digits bawaan sklearn (tanpa unduh) | 1 unit |
| 2 | File pendamping | Praktikum 12 - JST dan Deep Learning.ipynb (dari LMS) | 1 file |
| 3 | Internet (opsional) | Hanya untuk pengayaan Keras/TensorFlow (Kegiatan pengayaan; tidak wajib) | — |

## B.3 🛠 Langkah Kerja

🎨 **Arti warna pada kode:** ■ komentar (# ...) ■ kata kunci (if, for, def) ■ teks / string ■ angka ■ nama fungsi (print, len) ■ variabel

### Kegiatan 1 — Perceptron yang Belajar Sendiri (±40 menit)

```
# Gerbang AND: target 1 hanya jika kedua masukan 1
data = [([0, 0], 0), ([0, 1], 0), ([1, 0], 0), ([1, 1], 1)]

w = [0.0, 0.0]   # bobot awal: belum tahu apa-apa
b = 0.0
laju = 0.1       # seberapa besar 'geser' tiap kali salah

for epoch in range(10):
    salah = 0
    for x, target in data:
        tebakan = 1 if w[0]*x[0] + w[1]*x[1] + b > 0 else 0
        error = target - tebakan
        if error != 0:
            salah += 1
            w[0] += laju * error * x[0]
            w[1] += laju * error * x[1]
            b += laju * error
    print(f"epoch {epoch+1}: salah {salah}, w={w}, b={b:.1f}")
    if salah == 0:
        print("BELAJAR SELESAI -- semua benar!")
        break
```

| Baris Kode | Penjelasan |
|---|---|
| `data = [([0, 0], 0), ...]` | Empat contoh gerbang AND: (masukan, target). Perceptron akan menemukan sendiri aturan AND dari contoh — bukan dari IF yang kita tulis. |

| Baris Kode | Penjelasan |
|---|---|
| `w = [0.0, 0.0]; b = 0.0` | Berangkat dari nol besar: model belum tahu apa-apa. |
| `tebakan = 1 if ... > 0 else 0` | Neuron menyala jika jumlah berbobot melewati ambang — fungsi aktivasi undak (step). |
| `error = target - tebakan` | Bernilai +1 (harusnya menyala), −1 (harusnya diam), atau 0 (benar). |
| `w[0] += laju * error * x[0]` | Inti seluruh JST: salah → geser bobot sedikit ke arah yang benar. Versi raksasa dari aturan inilah (backpropagation) yang melatih ChatGPT. |
| `if salah == 0: break` | Konvergen: satu epoch penuh tanpa kesalahan. Untuk AND tercapai di epoch ke-6 dengan w=[0.2, 0.1], b=−0.2. |

### Kegiatan 2 — Tragedi XOR (±30 menit)

```python
# XOR: 1 jika masukan BERBEDA
data_xor = [([0, 0], 0), ([0, 1], 1), ([1, 0], 1), ([1, 1], 0)]

w = [0.0, 0.0]; b = 0.0
for epoch in range(50):
    salah = 0
    for x, target in data_xor:
        tebakan = 1 if w[0]*x[0] + w[1]*x[1] + b > 0 else 0
        error = target - tebakan
        if error != 0:
            salah += 1
            w[0] += laju*error*x[0]; w[1] += laju*error*x[1]; b += laju*error
print(f"Setelah 50 epoch XOR masih salah {salah} dari 4 -- MENYERAH.")
```

✅ **Cek hasil:** Hasil: 50 epoch pun perceptron tak pernah benar semua — bukan kurang lama, melainkan mustahil secara matematis (tak ada satu garis yang memisahkan XOR). Solusinya bukan berlatih lebih keras, melainkan menambah LAPISAN:

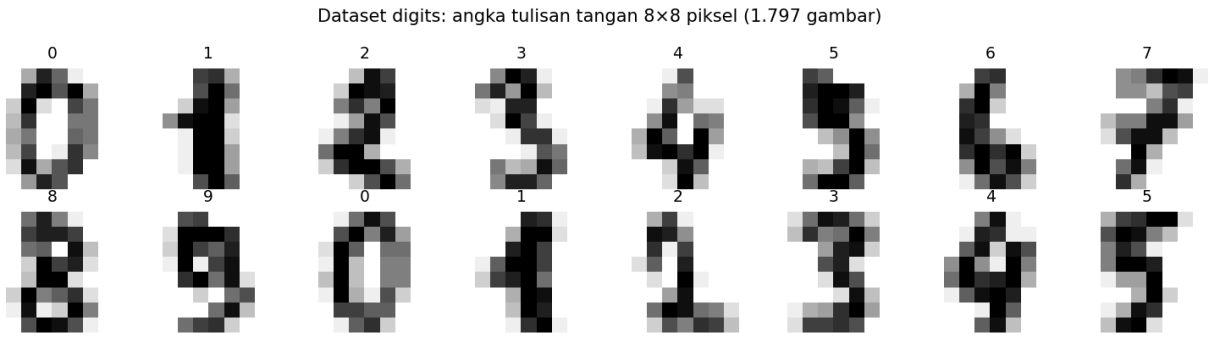
```python
from sklearn.neural_network import MLPClassifier

X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]
mlp_xor = MLPClassifier(hidden_layer_sizes=(4,), max_iter=3000,
                        random_state=1)
mlp_xor.fit(X, y)
print("Prediksi XOR:", mlp_xor.predict(X))   # [0 1 1 0] -- TERPECAHKAN
```

| Baris Kode | Penjelasan |
|---|---|
| `hidden_layer_sizes=(4,)` | Satu lapisan tersembunyi berisi 4 neuron — cukup untuk melengkungkan pemisah dan menaklukkan XOR yang 20 tahun membekukan riset AI. |
| `max_iter=3000` | Jaringan berlapis butuh iterasi belajar lebih banyak (backpropagation bekerja di baliknya). |

### Kegiatan 3 — MLP Mengenali Angka Tulisan Tangan (±90 menit)

Dataset digits bawaan scikit-learn: 1.797 foto angka tulisan tangan 8×8 piksel. Tiap gambar = 64 angka kecerahan — bagi jaringan, gambar hanyalah 64 masukan:




```python
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

digits = load_digits()
print("Bentuk data:", digits.data.shape)   # (1797, 64)

X_latih, X_uji, y_latih, y_uji = train_test_split(
    digits.data / 16.0, digits.target, test_size=0.25, random_state=42)

jaringan = MLPClassifier(hidden_layer_sizes=(64,), max_iter=50,
                         random_state=42)
jaringan.fit(X_latih, y_latih)
print("Akurasi latih:", round(jaringan.score(X_latih, y_latih), 3))
print("Akurasi uji  :", round(jaringan.score(X_uji, y_uji), 3))
```

| Baris Kode | Penjelasan |
|---|---|
| `digits.data.shape → (1797, 64)` | 1.797 gambar, masing-masing sudah “diluruskan” menjadi 64 angka piksel (8×8). Bagi jaringan, mengenali gambar = klasifikasi 64 fitur. |
| `digits.data / 16.0` | Normalisasi: kecerahan 0–16 disetarakan ke 0–1. Jaringan syaraf belajar jauh lebih stabil bila semua masukan berskala serupa. |
| `hidden_layer_sizes=(64,)` | Arsitektur: 64 masukan → 64 neuron tersembunyi → 10 keluaran (digit 0–9). Total ribuan bobot yang disetel otomatis. |
| `score latih vs uji` | ± 0,973 vs 0,962 — tinggi dan berdekatan: belajar sungguhan, bukan menghafal (bandingkan pelajaran P10!). |

```python
# bedah kesalahan: gambar mana yang membuat jaringan tertipu?
prediksi = jaringan.predict(X_uji)
salah_idx = [i for i in range(len(y_uji)) if prediksi[i] != y_uji[i]]
print("Jumlah salah:", len(salah_idx), "dari", len(y_uji))

for urut, i in enumerate(salah_idx[:6]):
    plt.subplot(1, 6, urut + 1)
    plt.imshow(X_uji[i].reshape(8, 8), cmap="gray_r")
    plt.title(f"asli {y_uji[i]} / tebak {prediksi[i]}", fontsize=8)
    plt.axis("off")
plt.show()
```

| Baris Kode | Penjelasan |
|---|---|
| `salah_idx = [i for i in ... if prediksi[i] != y_uji[i]]` | List comprehension menyaring indeks gambar yang salah ditebak. |
| `X_uji[i].reshape(8, 8)` | Mengembalikan 64 angka menjadi kotak 8×8 agar bisa digambar ulang. |

| Baris Kode | Penjelasan |
|---|---|
| `plt.title(f"asli ... / tebak ...")` | Lihat sendiri: angka yang salah tebak biasanya memang ambigu — 4 mirip 9, 3 mirip 8. Manusia pun sering ragu. |

### Kegiatan 4 — Pengayaan (Opsional): Mencicipi Keras/TensorFlow

Bagi yang laptopnya kuat (RAM ≥ 8 GB) dan ingin mencicipi tool industri: pasang TensorFlow (pip install tensorflow, ±600 MB), lalu pola yang sama persis di Keras terlihat seperti ini (cukup dibaca jika tidak menginstal):

**Pengayaan — Keras (dibaca saja jika tidak instal)**

```
from tensorflow import keras
model = keras.Sequential([
    keras.layers.Dense(64, activation="relu", input_shape=(64,)),
    keras.layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(X_latih, y_latih, epochs=20)
```

*Perhatikan: konsepnya identik dengan MLPClassifier — lapisan Dense, aktivasi, fit. Menguasai konsep hari ini berarti tinggal ganti sintaks saat industri meminta Keras/PyTorch.*

### Kegiatan 5 — ✍️ LATIHAN MANDIRI (±30 menit + dilanjutkan di rumah)

1. Latihan 1 — Latih perceptron dari nol untuk gerbang OR (target 1 jika salah satu masukan 1). Berapa epoch sampai konvergen? Bandingkan bobot akhirnya dengan AND.
2. Latihan 2 — Eksperimen arsitektur digits: coba hidden_layer_sizes (16,), (64,), (64, 32), dan max_iter 10 vs 50. Buat tabel akurasi latih vs uji; konfigurasi mana terbaik? Adakah tanda overfitting?
3. Latihan 3 — Analisis (markdown, ≥4 kalimat): (a) mengapa XOR mustahil bagi perceptron tetapi mudah bagi MLP? (b) hubungkan dengan face unlock: mengapa mengenali wajah butuh jaringan yang “dalam”?

## B.4 📩 Menyimpan dan Mengumpulkan

| No | Langkah Kerja | Hasil yang Diharapkan |
|---|---|---|
| **1** | Kernel → Restart & Run All; pastikan gambar digit dan semua skor tampil. | Seluruh sel berjalan tanpa error. |
| **2** | Rename file: P12_NIM_Nama. | Contoh: P12_20260001_Budi.ipynb. |
| **3** | Unggah ke LMS sebelum pertemuan 13. | Status LMS “terkirim”. |

# IV. PENILAIAN

| Aspek | Baik (3) | Cukup (2) | Kurang (1) |
|---|---|---|---|
| Kode berjalan | Perceptron + MLP digits + latihan jalan semua | 1 bagian error | ≥2 error |
| Eksperimen arsitektur | Tabel akurasi beberapa konfigurasi + pengamatan | Pengamatan minim | Tidak dikerjakan |

| Aspek | Baik (3) | Cukup (2) | Kurang (1) |
|---|---|---|---|
| Analisis | XOR & kaitan deep learning dijelaskan logis | Analisis dangkal | Tidak ada |

*Nilai notebook = (skor perolehan / 9) × 100 — tercatat sebagai Tugas pertemuan 12 (bobot 2%).*

## V. KESIMPULAN

- Neuron buatan = jumlah berbobot + aktivasi; belajar = menggeser bobot sedikit demi sedikit setiap kali salah — prinsip yang sama dari perceptron 1958 sampai ChatGPT.
- Perceptron tunggal hanya sanggup memisahkan dengan satu garis lurus: AND/OR bisa, XOR mustahil (tragedi 1969 yang membekukan riset AI).
- Hidden layer melengkungkan pemisah; makin berlapis makin “dalam” (deep learning) — lapisan awal mengenali pola sederhana, lapisan lanjut merangkainya menjadi konsep.
- Bagi jaringan, gambar hanyalah deretan angka piksel: digits 8×8 = 64 masukan; normalisasi skala masukan membuat belajar stabil; akurasi uji ± 0,96 dengan kesalahan yang “manusiawi” (4 mirip 9).
- scikit-learn cukup untuk konsep dan data kecil; Keras/PyTorch adalah sintaks industrinya — konsepnya sudah kalian genggam.

## VI. PRATINJAU PERTEMUAN 13

Materi berikutnya: LLM dan Generative AI — teknologi di balik ChatGPT: cara kerjanya secara konseptual, seni menulis prompt yang efektif, memanggil LLM dari Python, dan etika penggunaannya. Praktik membangun chatbot FAQ kampus. Siapkan koneksi internet.

## VII. REFERENSI


- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408 — makalah kelahiran perceptron.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536 — backpropagation yang menghidupkan kembali JST.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444 (terindeks Scopus).
- Mulyono, dkk. (2024). A high accuracy of deep learning based CNN architecture: classic, VGGNet, and ResNet50 for Covid-19 image classification. TELKOMNIKA, 22(5) (terindeks Scopus) — riset CNN oleh peneliti Indonesia.
- Géron, A. (2022). Hands-On Machine Learning (3rd ed.) — Bab 10 (Introduction to ANN).
- TensorFlow Playground (playground.tensorflow.org) — mainkan hidden layer di browser, tanpa kode; 3Blue1Brown seri video “Neural Networks”.